

TAREA 2: SIMULANDO ELOTELTAG Y APLICACIÓN FIND MY

NOMBRES: JULIÁN NÚÑEZ, VICENTE MUJICA, VALENTINA SOTO Y VICENTE ANDRADE
PROFESOR: AGUSTÍN GONZÁLEZ VALENZUELA

Introducción

La presente tarea tiene como objetivo simular gráficamente el funcionamiento de un sistema de localización de objetos inspirado en la tecnología AirTag de Apple, denominado en este contexto EloTelTag.

El sistema ya realizado con los EloTelTags evoluciona hacia un simulador gráfico interactivo desarrollado con JavaFX, operando bajo el paradigma de "programación conducida por eventos". Los dispositivos (EloTelTags, celulares y tablets) ahora se desplazan físicamente sobre un territorio visualizado en pantalla, manteniéndose confinados al mapa y rebotando al alcanzar sus bordes. Mientras la simulación avanza las búsquedas de dispositivos se manifiestan gráficamente mediante un radar expansivo y la emisión de sonidos cortos. Asimismo, la funcionalidad Find My se integra directamente en la interfaz: los usuarios pueden presionar sobre la vista de sus equipos para desplegar un menú emergente y consultar una ventana que actualiza periódicamente y en tiempo real la posición de todos sus bienes registrados.

El desarrollo de esta tarea se estructura en cuatro etapas incrementales. La primera etapa abarca la lectura del archivo de configuración y el despliegue gráfico únicamente de los celulares con un menú inactivo. La segunda incorpora las clases para EloTelTags y tablets, habilitando la simulación de movimiento de todos los equipos mediante el menú. La tercera añade la clase ETnube para el reporte de posiciones, las señales visuales de radar y la visualización estática del menú FindMy. Finalmente, la cuarta etapa completa todas las funcionalidades dinámicas del sistema (sonidos y actualizaciones de interfaz) e integra la documentación del código usando Javadoc.

El programa recibe como entrada un archivo de configuración. A diferencia del trabajo anterior, este programa no entrega un archivo de salida, sino que genera una aplicación gráfica de JavaFX, la cual muestra las posiciones de los dispositivos en un mapa y cómo estos se mueven.

Stage 4

Interacción entre clases y Arquitectura MVC

El diseño de la aplicación se rige estrictamente por el patrón Modelo-Vista-Controlador (MVC) adaptado al entorno de JavaFX y la programación conducida por eventos.

- **El Modelo:** Las clases lógicas (**Equipo**, **Cellular**, **EloTelTag**, **Tablet**) encapsulan el estado del sistema. El aspecto fundamental de su implementación es el uso de propiedades observables de JavaFX (**SimpleDoubleProperty**) para las coordenadas espaciales (x e y). Esto permite que el estado cambie sin conocer ni depender de cómo se dibujará gráficamente. Adicionalmente, la clase **ETNube** actúa como un modelo de datos centralizado (Singleton de facto) que almacena los reportes de ubicación.
- **La Vista:** Las clases gráficas (**CellularView**, **EloTelTagView**) construyen la interfaz extendiendo la clase **Group** de JavaFX. En lugar de actualizar manualmente la posición de los polígonos fotograma a fotograma, las vistas utilizan el método **.bind()** para acoplar las coordenadas de sus formas geométricas directamente a las propiedades observables del modelo correspondiente.
- **El Controlador:** El control principal recae sobre la clase **Stage4** y **Territory**, donde reside el **Timeline** principal. Este bucle de eventos dicta el ritmo de la simulación iterando sobre el método **move()** de los modelos. A su vez, vistas como **CellularView** actúan como controladores locales manejando eventos de usuario (**setOnMouseClicked**) para desplegar ventanas emergentes y lanzar **Timelines** secundarios que actualizan la información de *Find My* cada un segundo.

Dificultades

Durante el desarrollo de la Etapa 4, el equipo se enfrentó a los siguientes desafíos técnicos relacionados con la implementación en Java:

1. **Sincronización de eventos y concurrencia UI:** Implementar la actualización de la ventana emergente *FindMy* requirió aislar la ejecución para no bloquear el hilo principal de JavaFX. La dificultad radicó en crear un **Timeline** secundario dentro de **CellularView** que consultara a ETNube cada un segundo, asegurando que este reloj interno se detuviera correctamente (`updateTimeline.stop()`) al cerrar el popup (`setOnHidden`) para evitar fugas de memoria (Memory Leaks).
2. **Manejo de Transiciones y Animaciones (RadarPing):** Replicar la expansión del radar de los dispositivos exigió comprender el uso de transiciones paralelas. Se debió combinar **ScaleTransition** y **FadeTransition** para que el círculo de búsqueda creciera y se desvaneciera simultáneamente. Un reto particular fue asegurar la eliminación del nodo gráfico una vez finalizada la animación (`setOnFinished`) para no sobrecargar el **Pane** con objetos invisibles residuales.
3. **Gestión de colisiones y enrutamiento trigonométrico:** Traducir el rebote de los equipos en los bordes del **TerritoryView** a lógica matemática en la clase **Equipo** requirió un ajuste cuidadoso del ángulo de movimiento (`theta`). Se dificultó inicialmente identificar que, para invertir la componente perpendicular de la velocidad al rebotar en el eje X, el ángulo debía actualizarse a $\pi - \theta$, y para el rebote en el eje Y, a $-\theta$, manteniendo el comportamiento aleatorio dictado por `dtheta`.
4. **Integración de recursos multimedia (AudioClip):** Para cumplir con el requerimiento de emitir un sonido corto durante la expansión del radar, se utilizó la clase **AudioClip** de JavaFX. El desafío principal consistió en instanciar el archivo de audio una única vez en el método principal (`start`) y no en cada llamada al método `spawnRadar`, con el fin de evitar la saturación del *Garbage Collector* y asegurar que los múltiples "pings" concurrentes se reprodujeran sin latencia.

Diagrama UML

Como se observa en el diagrama de clases de la Figura 1, la estructura del software evidencia una separación clara de responsabilidades bajo el patrón de diseño Modelo-Vista-Controlador (MVC).

En el núcleo del modelo se posiciona la clase base **Equipo**, la cual encapsula los atributos físicos y matemáticos comunes de los dispositivos, tales como la velocidad, el ángulo de desplazamiento y la lógica trigonométrica necesaria para calcular los rebotes en los límites del mapa establecidos por **TerritoryView**. La característica fundamental de **Equipo** es la definición de sus coordenadas mediante objetos **DoubleProperty** (**SimpleDoubleProperty**), exponiendo los métodos observables **xProperty()** y **yProperty()**. Las subclases especializadas, **Cellular** y **EloTelTag**, extienden esta funcionalidad base; por ejemplo, **Cellular** añade la capacidad de conectarse e interactuar con la infraestructura de **ETNube** para actualizar de forma centralizada la posición global reportada por los rastreadores.

Por otro lado, la capa de la Vista se desmarca por completo del cómputo algorítmico del movimiento. Las clases **CellularView** y **EloTelTagView** extienden la clase **Group** de JavaFX e instancian componentes geométricos primitivos (**Rectangle** y **Circle**, respectivamente) junto con elementos de texto para las etiquetas de los usuarios. En lugar de implementar métodos imperativos de actualización periódica, el posicionamiento de estos nodos visuales se delega al motor de bindings de JavaFX. Utilizando expresiones como **circle.centerXProperty().bind(tag.xProperty())**, cualquier mutación en los hilos o estados del modelo se refleja inmediatamente en el lienzo gráfico en tiempo real, garantizando un acoplamiento débil y una alta cohesión en el diseño del software.

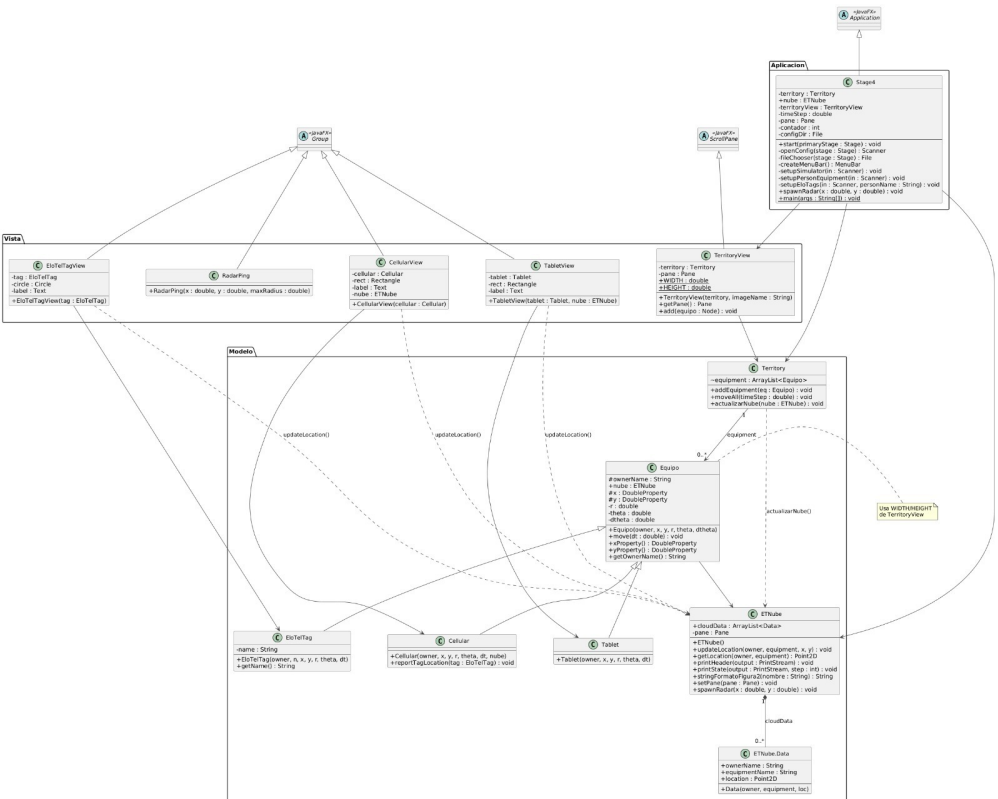


Figure 1: Diagrama de clases UML

Extra-crédito: Implementación de GFindMy

Como parte de los requerimientos extendidos, el equipo implementó exitosamente la funcionalidad *GFindMy*. Al interactuar con el menú emergente de un dispositivo, esta opción instancia una ventana **Stage** independiente que reutiliza la imagen satelital del territorio como fondo sobre un **ScrollPane**. El comportamiento dinámico de esta vista se gestiona mediante un **Timeline** aislado que, cada un segundo, consulta el modelo centralizado **ETNube** y renderiza nodos gráficos primitivos en las últimas coordenadas conocidas de los bienes de la persona. Este enfoque demuestra la capacidad del sistema para instanciar múltiples vistas independientes que consumen la misma fuente de verdad en la arquitectura MVC.